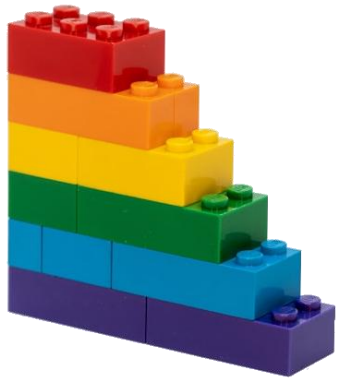




Reusability in IaC

The “Art” 🖼️ 🎨 🖌️ of Module development in Terraform

(by Stefan Rapp, 2nd July 2025, 16:00 – 16:50)



```
resource "azurerm_role_assignment" "before" {
  for_each
  name
  description
  scope
  role_definition_id
  role_definition_name
  principal_id
  principal_type
  condition
  condition_version
  delegated_managed_identity_resource_id
  skip_service_principal_aad_check

  = var.role_assignments_before != null ? var.role_assignments_before : {}
  = each.key
  = each.value.description
  = try(var.role_assignment_scope_ids[each.value.scope_name], each.value.)
  = each.value.role_definition_id
  = each.value.role_definition_name
  = try(local.role_assignment_possible_principal_ids[each.value.principal
  = each.value.principal_type
  = each.value.condition
  = each.value.condition_version
  = each.value.delegated_managed_identity_resource_id
  = each.value.skip_service_principal_aad_check

months ago | 1 author

azurerm_machine_learning_workspace "this" {
  = [azurerm_role_assignment.before]
  = var.create_machine_learning_workspace ? 1 : 0
  = var.machine_learning_workspace_name
  = try(azurerm_resource_group.this[0].name, var.resource_group_name)
  = var.module_location
  = var.machine_learning_workspace_application_insights_id
  = var.machine_learning_workspace_key_vault_id
  = var.machine_learning_workspace_storage_account_id
  = var.machine_learning_workspace_container_registry_id
  = var.machine_learning_workspace_public_network_access_enabled
  = var.machine_learning_workspace_image_build_compute_name
  = var.machine_learning_workspace_description
  = var.machine_learning_workspace_friendly_name
  = var.machine_learning_workspace_high_business_impact
  = var.machine_learning_workspace_possible_identity_ids[var.machine_learning
  = var.machine_learning_workspace_v1_legacy_mode_enabled
  = var.machine_learning_workspace_sku_name

!= null ? var.machine_learning_workspace_encryptions :

key_vault_id, encryption.value.key_vault_id
key_id != null ? encrypt
possible_identity_ids[encryption.value.use
```

Table of contents

1. Baseline Knowledge
2. Using Modules in Terraform
3. Best Practices (“*Dos & Don’ts*”)
4. Publishing & Versioning
5. Additional Tooling
6. Key Takeaways (Q&A)



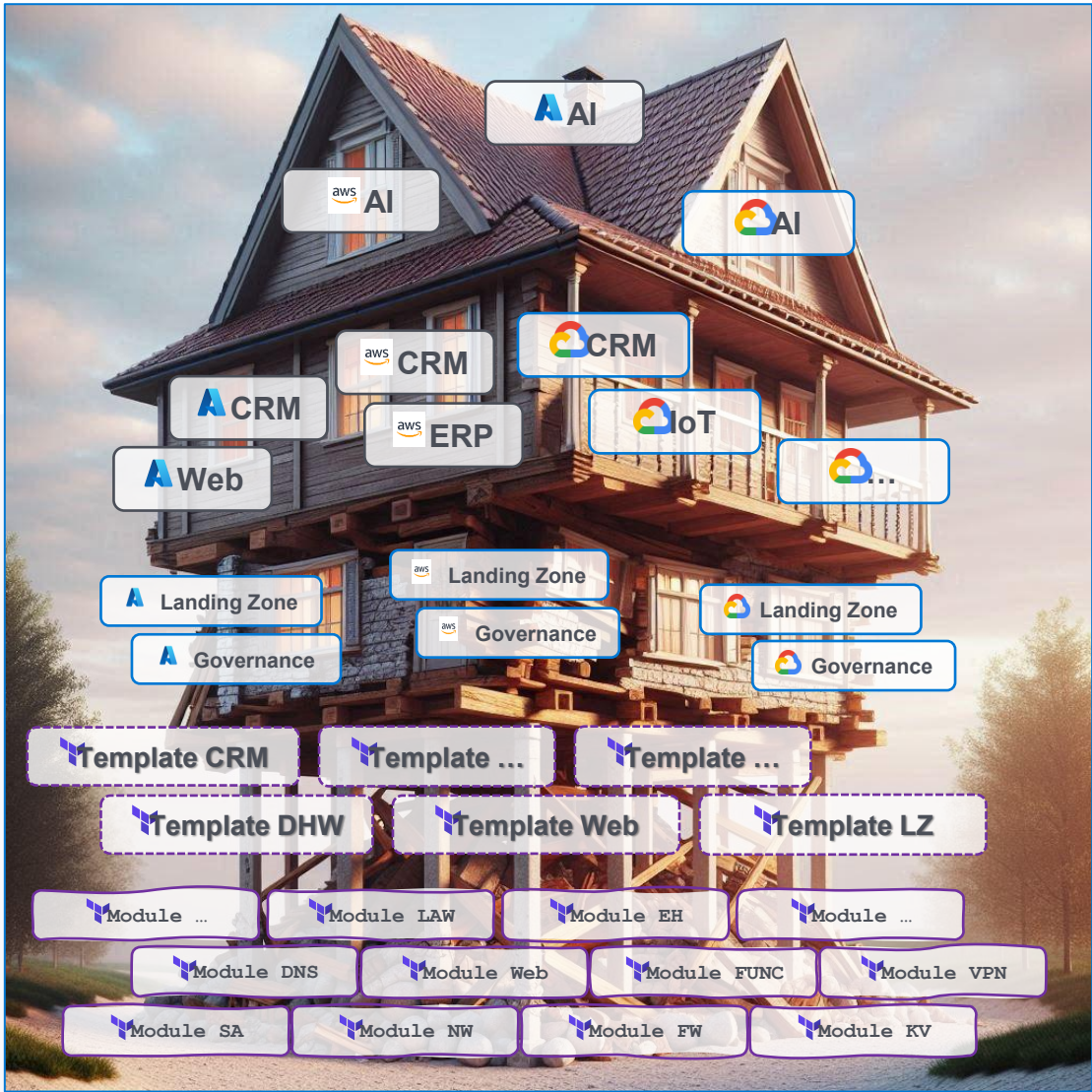


Why is modularization so essential?

From plane #code → to modules!

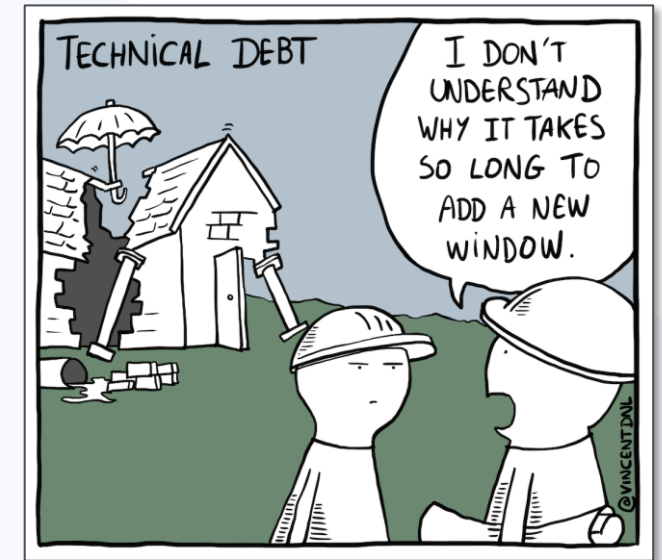
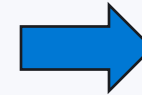
Baseline Knowledge

Why Modules are essential in IaC?



Consequences of bad modules?

→ After the first change:

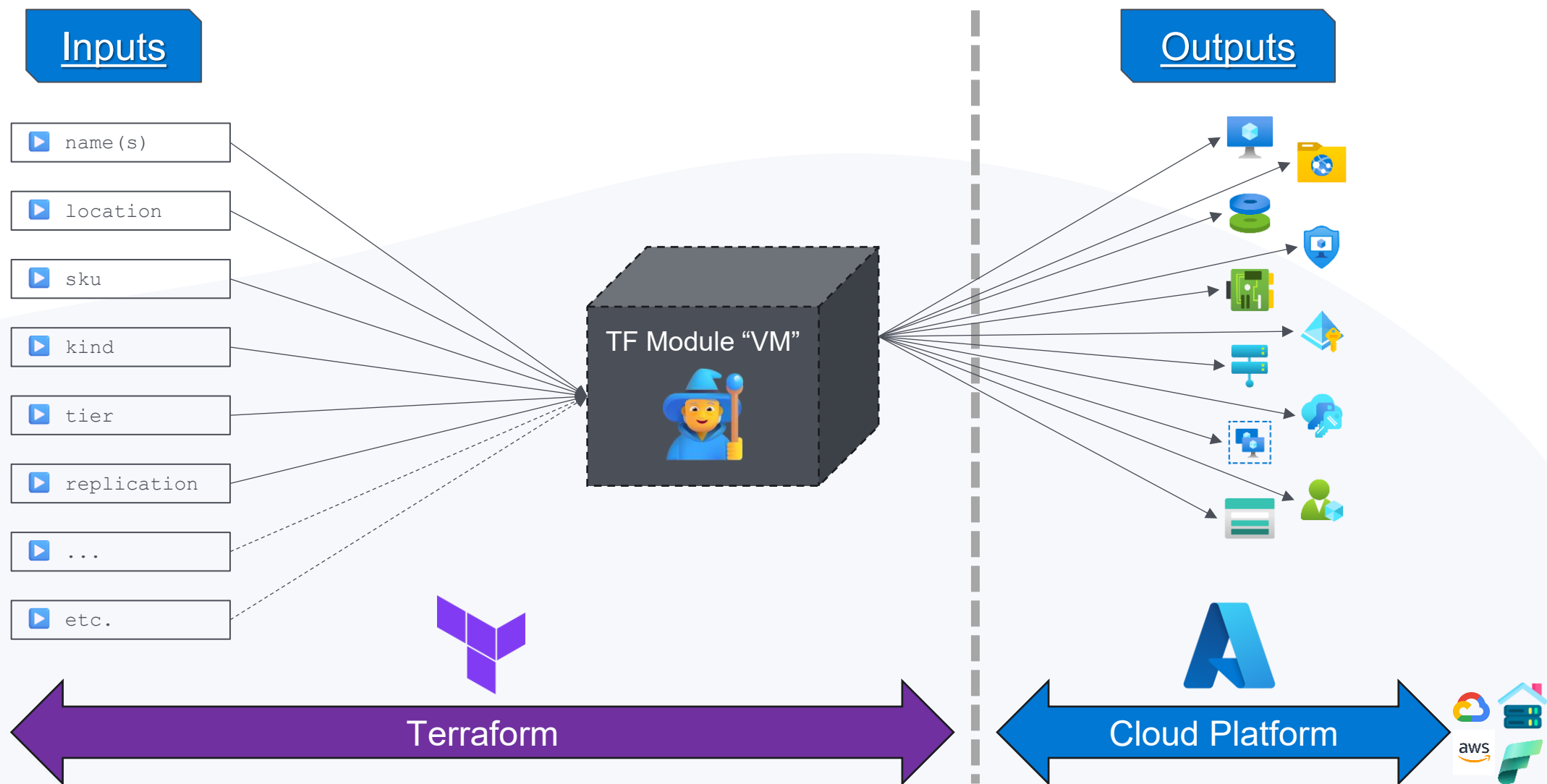




What does **modularization** mean Terraform?

Using Modules in Terraform

Accepts inputs & produces outputs



A module in Terraform is...

the **definition** of infrastructure (“What?”). 🏛️

independently **usable** & **testable**. 🧑‍🔬

reusable across different **projects** or **environments**. 🌴

separated in code repositories. 🏗️

used to **encapsulate** “complex” logic. 🧑‍🔧

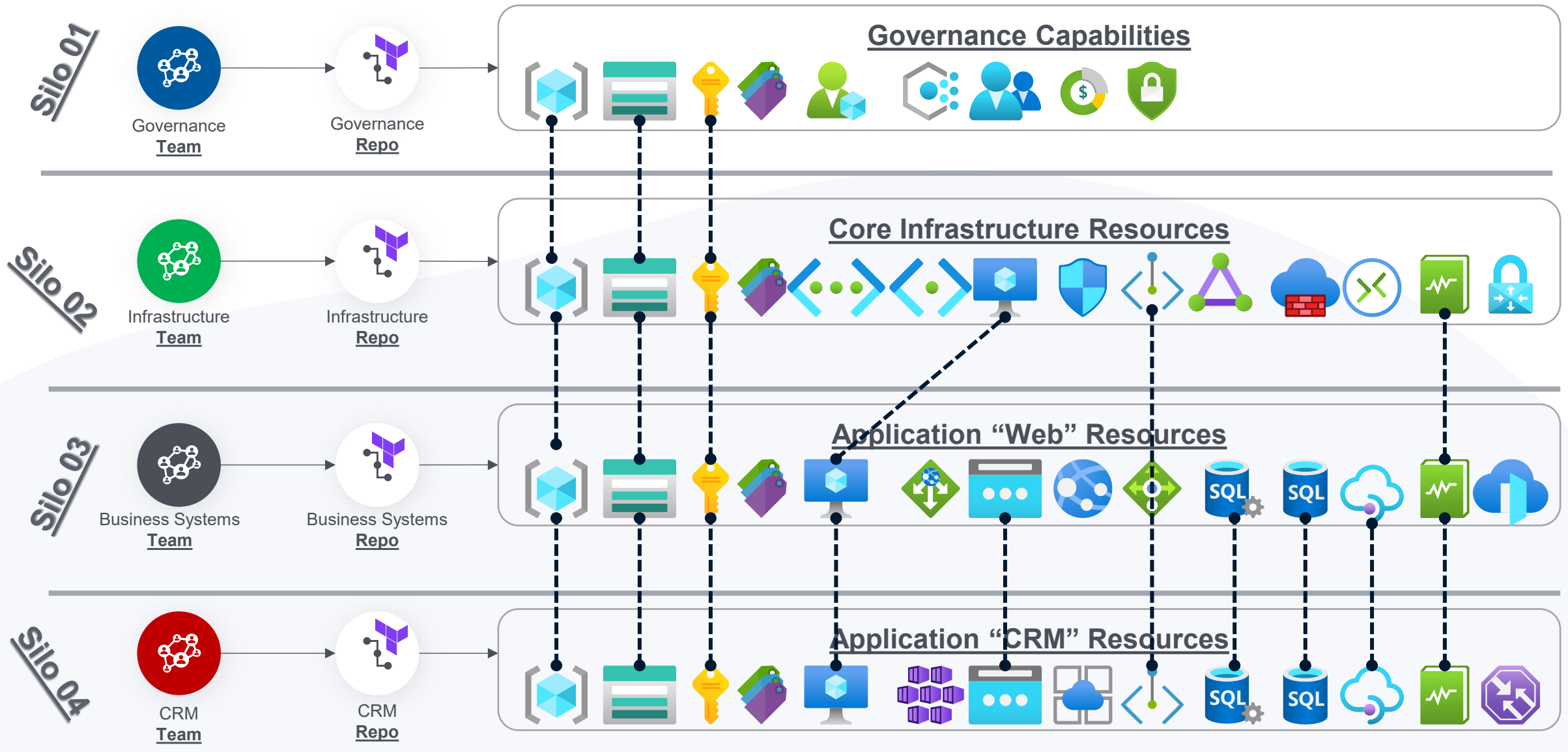
is **easier** to manage, test, update and consume. 🔄

“A module is a container for multiple resources that are used together.”

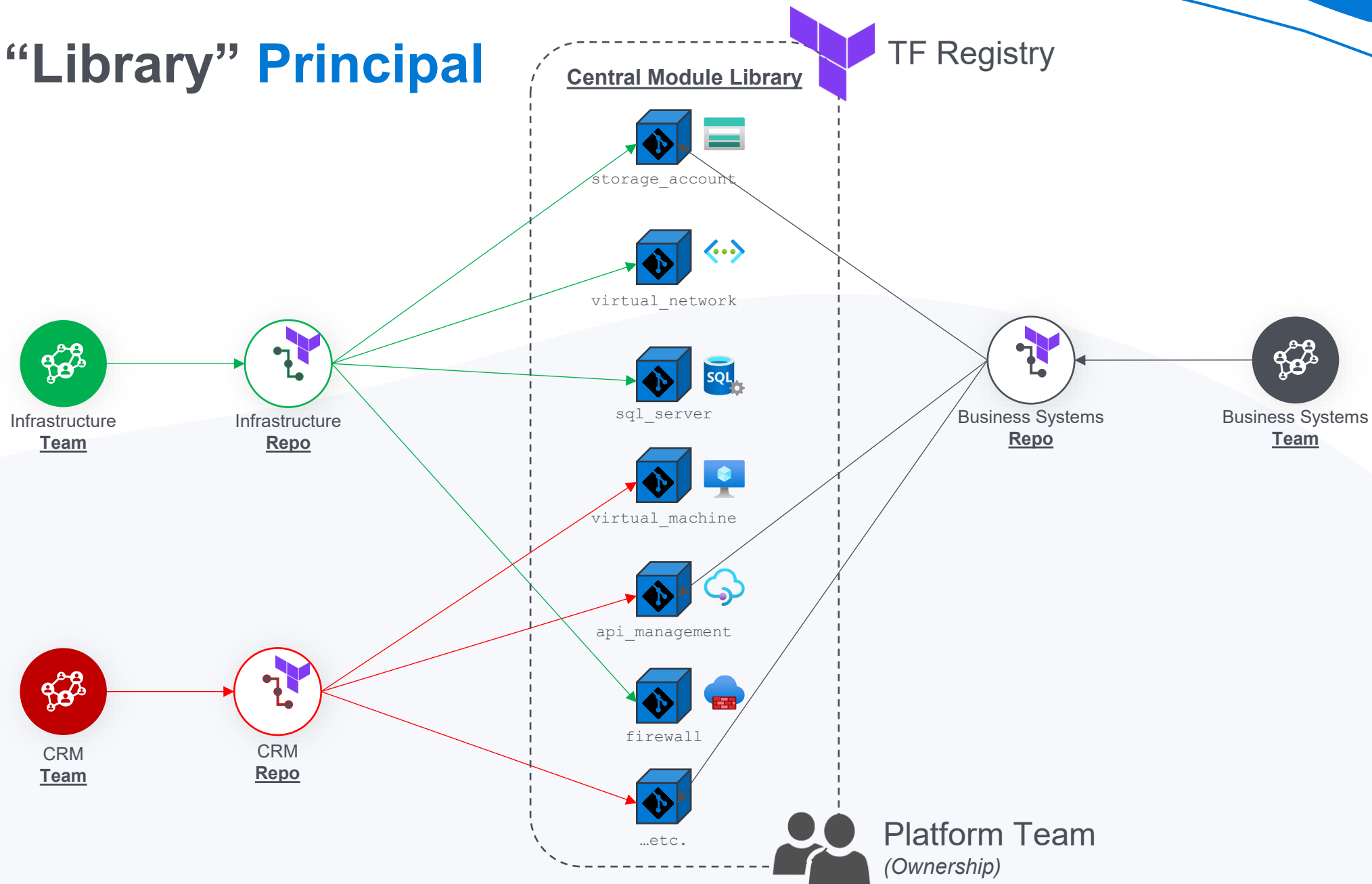
“A collection of Terraform files and folder in a repo.”

“Is the baseline for producing quality and reusable Terraform code.”

Current Situation in organization



The “Library” Principal





How does a **Module** look like in `#Code`?

DEMO Time!

Standardized Module Structure

1.

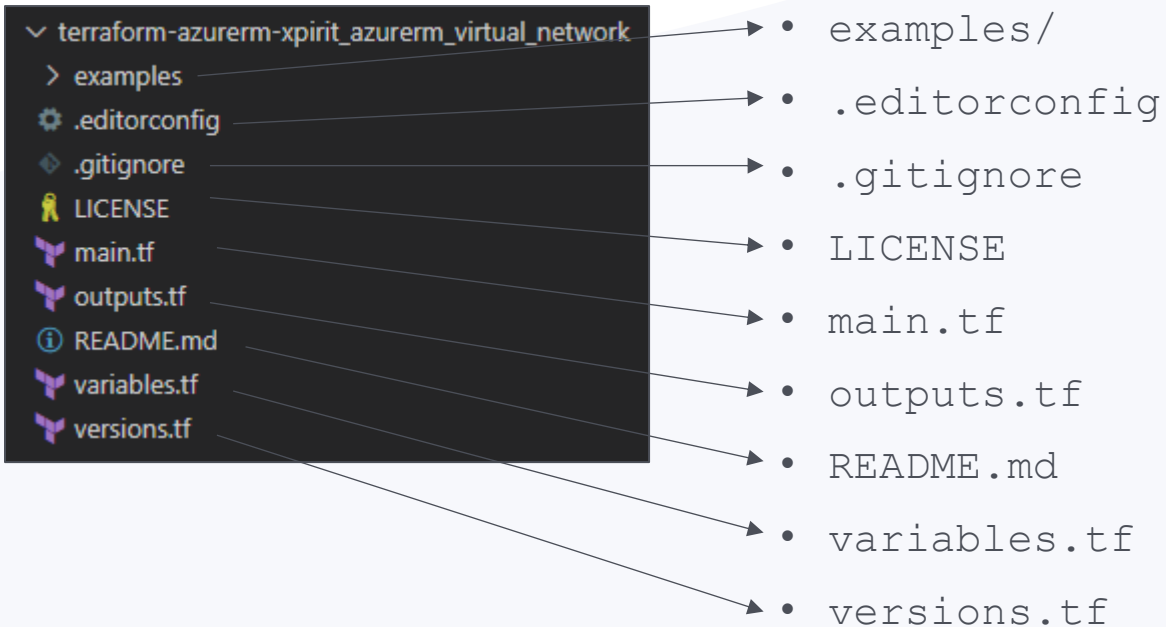
file & directory layout

2.

Distributed in separate
repositories

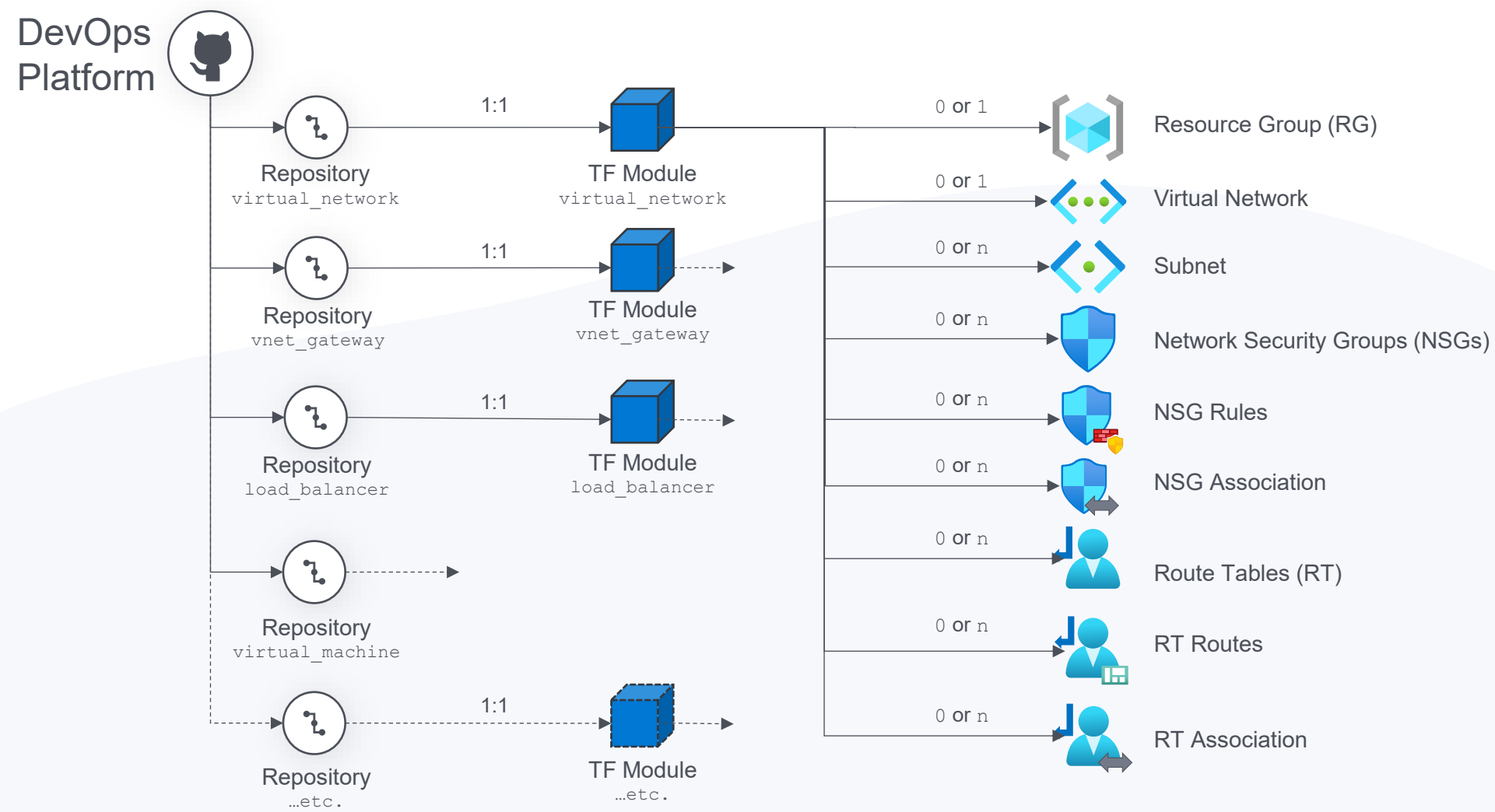
3.

Structure to generate
documentation



- separate subdirectory for each example incl. README.md
- maintain consistent code style ([Example](#))
- specifies untracked files that Git should ignore ([Example](#))
- which this module is available (publicly, privately)
- primary entry point, containing all needed resources
- to return results to the calling module
- what the module should be used for (basic documentation)
- (declaration) to accept values from the calling module.
- (constraint) define supported terraform & provider version

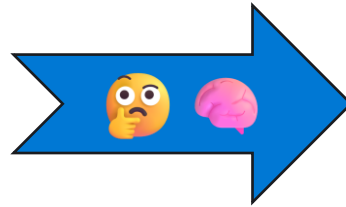
Standard Module Architecture



Ask yourself... !?



#1 Question



#2 Question

“What characterizes a...

- ***Virtual Network?***
- ***Storage Account?***
- ***Virtual Machine?***
- ...
- *etc.*

“How should the module look like so it can be used by several organizations, BUs, departments or workloads?”

The “triple knowledge gap” problem

“Writing modules is an **easy**, **dull** and **repetitive** task! 🤒

Let our **trainees** do the job!” 🧑



How to design reusable Modules? 🎨

- Structure?
- Patterns & Best Practices?
- Breaking Changes
- Lifecycle handling?

TF Module

📦 #code

TF Provider



Cloud Platform



How Terraform sees the world? 🧐

- Attributes?
- Dependencies?
- Version updates?
- Resource Behavior?

How the Cloud works under the hood? 🚗

- Resource configurations?
- Interdependencies?
- Limits, Quotas, Hidden Behaviors?
- Policies?



Use **Azure Verified Modules** (AVM) as Starting Point 🚗



What are the “Dos & Don’ts” regarding modules?

Best-Practices

Distinguish Bad from Good Modules



Bad Modules 🤔

- ... can drive you **nuts**. 🤪
- ... are **horrible** to use. 🤢
- ... lead to bad **results**. 🤔



Good Modules 😊

- ... are **fun** to use. 🤪
- ... lead to **great** results. 👍
- ... are **easy** to maintain and manage. 🛠️

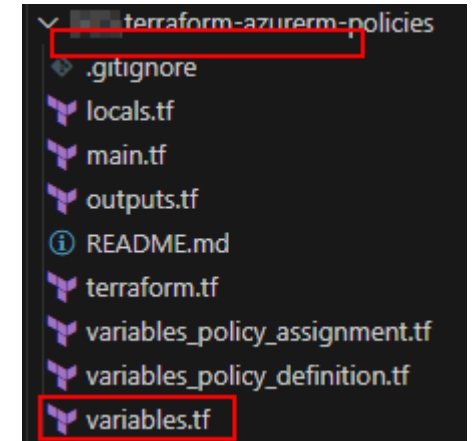
➔ Details: [The Art of Modularization in Terraform – Mr. Azure](#) 🖼️ 🎨 🖋️

Follow the “KISS” 🗨️ principal



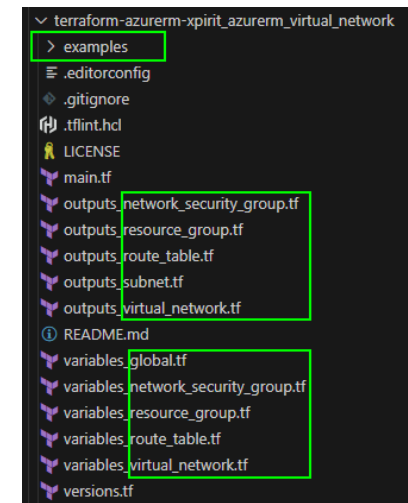
Bad Practice 🙄

- ❌ Big monster 🐉 repos with multiple modules 📁 → 📁 → 📁 → ...
- ❌ Missing `examples` folder
- ❌ Unstructured & hard to understand
- ❌ “Endless” `variables.tf` or `outputs.tf` files



Good Practice 👍

- ✅ Each module is a separate **repo** → Define the **file & directory** layout.
- ✅ Philosophy → “*Doing one thing and doing it well*”. (single concern!)
- ✅ Small & self-contained 📦
- ✅ Serve a specific purpose → Discrete & specialized
- ✅ Independently testable 🧪 → `examples` folder
- ✅ Doing too much → ✂️ 🔪 Breaking it into smaller, more specialized modules.



Repo & Content Structure



Bad Practice 🙄

- ✗ No unique repo **naming**
- ✗ No standardized **structure & format**
- ✗ `data.tf` in modules
- ✗ No formatting & linting implemented

```
▼ Terraform_Modules
> terraform-azurerm-networkgateway
> terraform-azurerm-networksecuritygroup
> terraform-azurerm-policies
> terraform-azurerm-rbac-rngmt
> terraform-azurerm-virtualnetwork

variable "network_hub" {
  description = "(Mandatory) Is the Landing Zone network Hub?"
  type        = bool
  default     = false
}
```

```
▼ terraform-azurerm-virtualnetwork
.editorconfig
.gitignore
.terraform-docs.yml
data.tf
main.tf
only_hub.tf
only_spoke.tf
output.tf
README.md
terraform.tf
variables.tf
```



Good Practice 👍

- ✓ For orientation use [Standard Module Structure \(TF\)](#).
- ✓ Use a predefined repo **template**.
- ✓ Check, if all **files/folders** exist and **named** correctly. 🤖
- ✓ Validate in the pipeline before the **merge**.
- ✓ Use that structure to generate **documentation** (`README.md`).
- ✓ Avoid **nested** modules, due to complexity.

```
▼ terraform-azurerm-..._virtual_network
> examples
.editorconfig
.gitignore
.tflint.hcl
LICENSE
main.tf
output_route_table.tf
outputs_network_security_group.tf
outputs_resource_group.tf
outputs_subnet.tf
outputs_virtual_network.tf
README.md
variables_global.tf
variables_network_security_group.tf
variables_resource_group.tf
variables_route_table.tf
variables_virtual_network.tf
versions.tf
```

Proper Naming Convention



Bad Practice 🙄

- ✗ Each developer uses individual **names**.
- ✗ No standardized **abbreviations** or **prefixes**.
- ✗ Each module looks differently.
- ✗ Hard to troubleshoot.

```
resource "azurerm_subnet" "vpn-gw-subnet" {  
  name           = "GatewaySubnet"  
  resource_group_name = var.hub_resource_group_name  
  virtual_network_name = var.hub_vnet_name  
  address_prefixes   = [var.vpn_vnet_subnet_prefix]  
}  
  
# Associate gateway subnet to hub routing-tables  
data "azurerm_route_table" "route-table-hub" {  
  name           = var.hub_route_table_name  
  resource_group_name = var.hub_resource_group_name  
}  
  
resource "azurerm_subnet_route_table_association" "rt_association" {  
  subnet_id      = azurerm_subnet.vpn-gw-subnet.id  
  route_table_id = data.azurerm_route_table.route-table-hub.id  
}
```



Good Practice 👍

- ✓ Define the standard before the **start**.
- ✓ Stick to it!
- ✓ Agree on **naming** conventions, **abbreviations** and **prefixes**.
- ✓ Lint the code before the merge.
- ✓ Strive for **consistent** and **descriptive** names.

```
resource "azurerm_virtual_network_peering" "this" {  
  count           = var.create_virtual_network_peering ? 1 : 0  
  name           = var.virtual_network_peering_name  
  resource_group_name = try(azurerm_resource_group.this[0].name, var.resource_group_name)  
  virtual_network_name = try(azurerm_virtual_network.this[0].name, var.virtual_network_peering_virtual_network_name)  
  allow_forwarded_traffic = var.virtual_network_peering_allow_forwarded_traffic  
  allow_gateway_transit   = var.virtual_network_peering_allow_gateway_transit  
  allow_virtual_network_access = var.virtual_network_peering_allow_virtual_network_access  
  remote_virtual_network_id = var.virtual_network_peering_remote_virtual_network_id  
  triggers              = var.virtual_network_peering_triggers  
  use_remote_gateways    = var.virtual_network_peering_use_remote_gateways  
}  
  
resource "azurerm_subnet" "this" {  
  for_each = var.subnets != null ? var.subnets : {}  
  name     = each.key  
  resource_group_name = try(azurerm_resource_group.this[0].name, var.resource_group_name)  
  address_prefixes     = each.value.address_prefixes  
  private_endpoint_network_policies = each.value.private_endpoint_network_policies  
  service_endpoints     = each.value.service_endpoints  
  virtual_network_name = try(azurerm_virtual_network.this[0].name, var.virtual_network_name)  
}
```


Do not hardcode 📦 values



Bad Practice 🙄

- ❌ Do not use fixed patterns
- ❌ Avoid hard-coded values
- ❌ Not changeable
- ❌ Unflexible

```
resource "azurerm_virtual_network_gateway" "vpn-gateway" {  
  name           = "vgw-${var.location_code}-${var.name}-${var.environment}-${random_string.id.result}"  
  location       = var.location  
  resource_group_name = var.hub_resource_group_name  
  
  type      = "Vpn"  
  vpn_type  = "RouteBased"  
  
  active_active = false  
  enable_bgp    = false  
  sku           = "VpnGw1"  
  
  ip_configuration {  
    name                = "vnetGatewayConfig"  
    public_ip_address_id = azurerm_public_ip.vpn_public_ip.id  
    private_ip_address_allocation = "Dynamic"  
    subnet_id           = azurerm_subnet.vpn-gw-subnet.id  
  }  
}
```



Good Practice 👍

- ✅ Attributes mapped to a variable.
- ✅ Use default values in `variables.tf` file.
- ✅ default values are **overwritable**.

```
resource "azurerm_virtual_network_gateway" "this" {  
  count           = var.create_virtual_network_gateway ? 1 : 0  
  location        = var.module_location  
  resource_group_name = try(azurerm_resource_group.this[0].name, var.resource_group_name)  
  name           = var.virtual_network_gateway_name  
  active_active   = var.virtual_network_gateway_active_active  
  default_local_network_gateway_id = try(azurerm_local_network_gateway.this[var.virtual_network_gateway_name].id, null)  
  enable_bgp      = var.virtual_network_gateway_enable_bgp  
}
```

```
variable "virtual_network_gateway_type" {  
  description = "The type of the Virtual Network Gateway. Valid options are Vpn and ExpressRoute."  
  type        = string  
  default     = "Vpn"  
}  
  
variable "virtual_network_gateway_vpn_type" {  
  description = "The routing type of the Virtual Network Gateway. Valid options are RouteBased and PolicyBased."  
  type        = string  
  default     = "RouteBased"  
}
```

Keep modules “DRY” ☂️



Bad Practice 🙅

- ❌ Code duplications (same logic → multiple places)
- ❌ Reduced readability (more lines of code)
- ❌ Higher maintenance effort
- ❌ Scalability issues (“enterprise scale”)

```
resource "random_password" "datacenter" {  
  length      = 32  
  special     = true  
  override_special = "!"#$%&*()-_+=[]{}<>:?"  
}  
  
resource "random_password" "server" {  
  length      = 32  
  special     = true  
  override_special = "!"#$%&*()-_+=[]{}<>:?"  
}
```



Good Practice 👍

- ✅ Don't repeat yourself (DRY)
- ✅ Do not use a `resource` multiple times within the same module.
- ✅ Use `for_each` loops or `count` in module resources. ([Article](#))
- ✅ Try to use unique identifier `this` for the resource.

```
resource "azurerm_public_ip" "this" {  
  for_each = var.public_ips != null ? var.public_ips : {}  
  location = var.module_location  
  resource_group_name = try(azurerm_resource_group.this[0].name, var.  
  name = each.key  
  allocation_method = each.value.allocation_method  
  ddos_protection_mode = each.value.ddos_protection_mode  
  ddos_protection_plan_id = each.value.ddos_protection_plan_id
```



How to **manage** a module according to its lifecycle?

Publishing & Versioning

```

module_location = var.bastion_host_module_location
module_tags     = var.bastion_host_module_tags

# azurerm_resource_group
create_resource_group = var.create_bastion_host_resource_group
resource_group_name   = var.bastion_host_resource_group_name

# azurerm_public_ip
create_public_ip      = var.create_bastion_host_public_ip
public_ip_name        = var.bastion_host_public_ip_name
public_ip_domain_name_label = var.bastion_host_public_ip_name
public_ip_zones       = var.bastion_host_public_ip_zones

# azurerm_bastion_host
create_bastion_host = var.create_bastion_host
bastion_host_name   = var.bastion_host_name
# 4 weeks ago | 1 author (You)
bastion_host_ip_configurations = {
  # 4 weeks ago | 1 author (You)
  "2a8947-2efa-4b0e-ac61-c4086259a419" = {
    public_ip_address_id = null
    subnet_id            = module.virtual_network.subnet_ids["AzureBastion"]
  }
}

# private_dns_resolver
create_private_dns_resolver = var.create_private_dns_resolver
private_dns_resolver_name   = var.private_dns_resolver_name
private_dns_resolver_virtual_network_id = module.virtual_network.virtual_network_id

private_dns_resolver_inbound_endpoint = var.private_dns_resolver_inbound_endpoint
private_dns_resolver_inbound_endpoints = var.private_dns_resolver_inbound_endpoints
private_dns_resolver_inbound_endpoint_subnets = module.virtual_network.subnet_ids["private_dns_resolver_inbound_endpoint_subnets"]

private_dns_resolver_outbound_endpoint = var.private_dns_resolver_outbound_endpoint
private_dns_resolver_outbound_endpoints = var.private_dns_resolver_outbound_endpoints
private_dns_resolver_outbound_endpoint_subnets = module.virtual_network.subnet_ids["private_dns_resolver_outbound_endpoint_subnets"]

```

TF Cloud helps teams to **provision** infrastructure.

Support for **versioning** and list of **providers** and **modules**.

Handles **downloads** and **access** with TF Cloud API tokens.

Consumers do not need **direct** access to the module's repository.



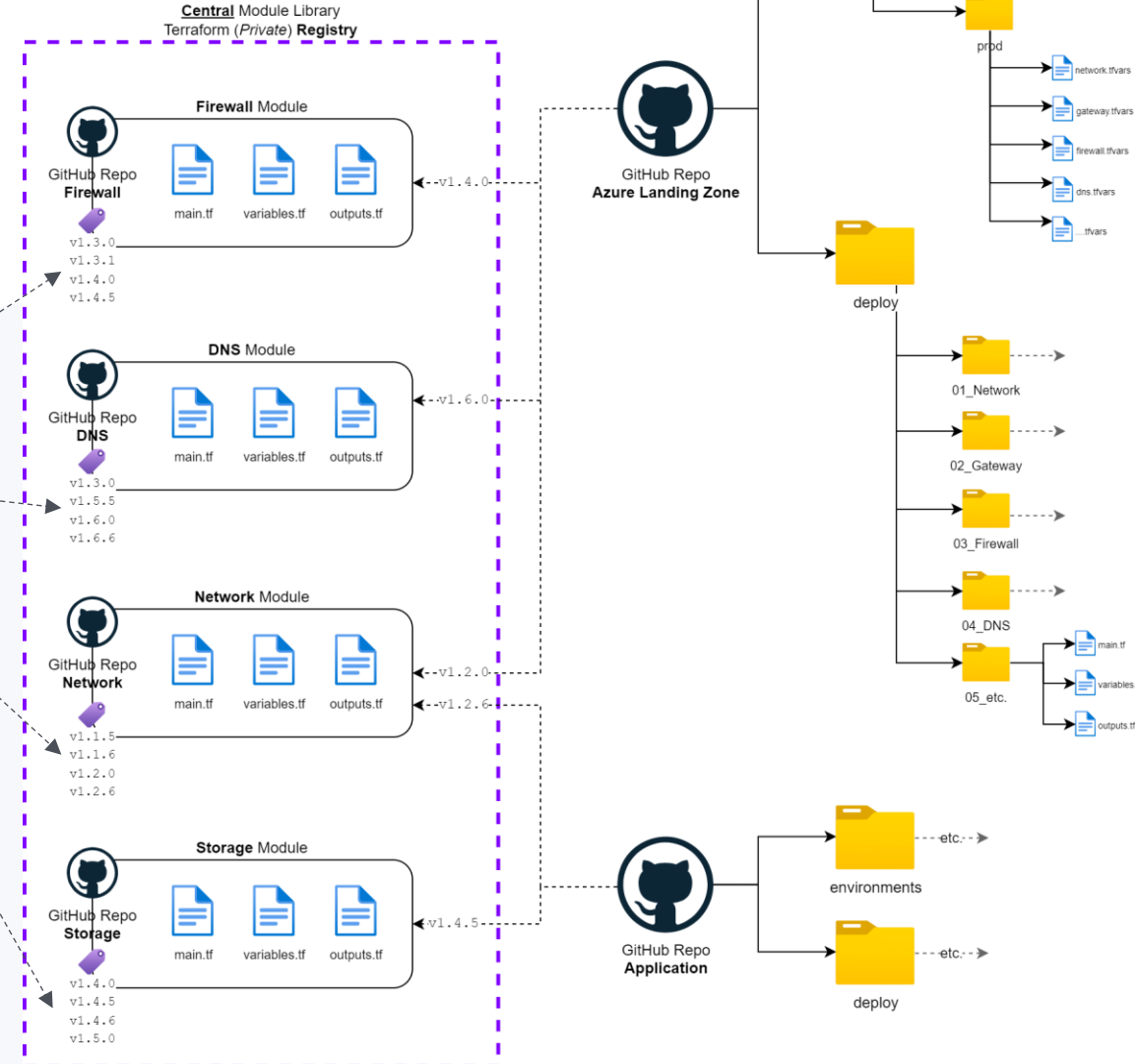


How does a **Registry** look like?

DEMO Time!

Reference Modules in Templates

Use [SemVer](#):



How to use modules in Templates?

- Local file system

```
module "dd34208a_7c40_4dd2_beb5_4a5bf4d21d22" {  
  source = "../../module-library-azurerm/terraform-azurerm-xpirit_azurerm_management_group"  
  
  create_management_group      = true  
  management_group_name       = "c06835d9-1135-4b19-b403-749c06443f4c"  
  management_group_display_name = "mg-pritix"  
  management_group_subscription_association_subscription_ids = []  
}
```

- Private registry

```
module "dd34208a_7c40_4dd2_beb5_4a5bf4d21d22" {  
  source = "app.terraform.io/Customer-ML-Pritix/xpirit_azurerm_management_group/azurerm"  
  version = "3.42.0-0.0.2"  
  
  create_management_group      = true  
  management_group_name       = "c06835d9-1135-4b19-b403-749c06443f4c"  
  management_group_display_name = "mg-pritix"  
  management_group_subscription_association_subscription_ids = []  
}
```

- Directly from the repository

```
module "dd34208a_7c40_4dd2_beb5_4a5bf4d21d22" {  
  source = "git::https://github.com/rapster83/terraform-azurerm-xpirit_azurerm_management_group?ref=v3.42.0-0.0.3"  
  
  create_management_group      = true  
  management_group_name       = "c06835d9-1135-4b19-b403-749c06443f4c"  
  management_group_display_name = "mg-pritix"  
  management_group_subscription_association_subscription_ids = []  
}
```

- Public registry (→ TF Registry)

```
module "dd34208a_7c40_4dd2_beb5_4a5bf4d21d22" {  
  source = "azrappster83/management_group/azurerm"  
  version = "4.0.0"  
  
  create_management_group      = true  
  management_group_name       = "c06835d9-1135-4b19-b403-749c06443f4c"  
  management_group_display_name = "mg-pritix"  
  management_group_subscription_association_subscription_ids = []  
}
```

Use [SemVer](#)



Which **tools** should be used to improve module quality?

Additional Tooling

Tools for Terraform

[tflint](#)

→ Linter for Terraform code + custom rules 📖

[checkov](#)

→ Code analysis (security & compliance) 🔒

[snyk](#) & [trivi](#)

→ Static code analysis 🔒

[terraform-docs](#)

→ Doc from input/output definitions 📄

[terratest](#)

(Go) → Framework for writing automated tests 🤖

[tfupdate](#)

→ Automates version updates ↻





What are the **essentials** of the session?

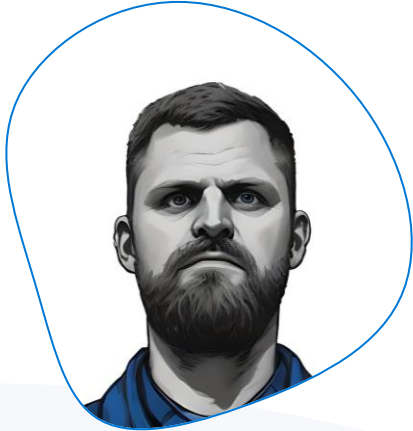
Key Takeaways

Key Takeaways

- ✔ Start Modularization from the **beginning!**  TOP
- ✔ Keep modules **flexible** (“*Do not hardcode*”). 
- ✔ Follow the code **best-practices**. 
- ✔ Make sure that all modules “*look the same*” (**standardization**). 
- ✔ Use a **central** module library across projects or departments. 
- ✔ Version your modules using **Tags**. 
- ✔ Use proper **Tools** to improve code quality. 



PROFILE – Speaker



Stefan Rapp

Cloud Solution Architect (CSA) & Microsoft MVP

Xebia

Let's engage: <https://www.linkedin.com/in/rapster83/>

#AzureRocks 🙌 🧑💻 🎸



E-Mail: info@blog.misterazure.com

Blog: <https://blog.misterazure.com>

GitHub: [@rapster83](https://github.com/rapster83)

